

MINGGU 3 — INFORMED (HEURISTIC) SEARCH STRATEGIES

B.1 Tujuan Praktikum

- Mahasiswa mampu mendefinisikan dan menggunakan fungsi heuristik $h(n)$ dalam program pencarian.
- Mahasiswa mampu mengimplementasikan Greedy Best-First Search dan A* Search.
- Mahasiswa mampu menjelaskan konsep admissibility dan consistency pada heuristik, serta membandingkan efisiensi informed search terhadap uninformed search yang telah dipelajari pada Minggu 2.

B.2 Dasar Teori Singkat

Informed (heuristic) search memanfaatkan fungsi heuristik $h(n)$ — estimasi biaya termurah dari node n menuju goal — untuk mengarahkan pencarian agar lebih efisien (AIMA Bab 3.5). Dua algoritma utama yang dipelajari pada sesi ini:

- Greedy Best-First Search — menggunakan evaluation function $f(n) = h(n)$. Selalu memilih node yang tampak paling dekat dengan tujuan, sehingga cepat namun tidak menjamin solusi optimal karena mengabaikan biaya yang sudah ditempuh ($g(n)$).
- A* Search — menggunakan evaluation function $f(n) = g(n) + h(n)$, menggabungkan biaya yang sudah ditempuh (g) dan estimasi biaya tersisa (h). Jika heuristik yang digunakan admissible (tidak pernah overestimate), A* dijamin cost-optimal.

Sebuah heuristik dikatakan admissible apabila untuk setiap node n berlaku $h(n) \leq h^(n)$, dengan $h^*(n)$ adalah biaya sebenarnya dari n menuju goal. Heuristik straight-line distance yang digunakan pada percobaan ini bersifat admissible karena jarak garis lurus tidak mungkin lebih jauh daripada jarak tempuh sesungguhnya di jalan.*

B.3 Alat & Bahan

- Python 3.8 ke atas, Jupyter Notebook / Google Colab
- Modul heapq (sama seperti Minggu 2)
- Variabel graph dari Percobaan 1 Minggu 2 (jalankan kembali Kode Program 1 sebelum melanjutkan)

B.4 Langkah Percobaan

Percobaan 1 — Mendefinisikan Fungsi Heuristik

Kita menggunakan heuristik jarak garis lurus (straight-line distance) menuju kota Bucharest, sesuai Figure 3.16 pada AIMA.

Kode Program 5 — Fungsi Heuristik h_SLD

```
# Fungsi heuristik: estimasi jarak garis lurus (straight-line distance)
# dari setiap kota menuju Bucharest (AIMA Fig. 3.16)
h_sld = {
    'Arad': 366, 'Bucharest': 0, 'Craiova': 160, 'Drobeta': 242,
    'Eforie': 161, 'Fagaras': 176, 'Giurgiu': 77, 'Hirsova': 151,
    'Iasi': 226, 'Lugoj': 244, 'Mehadia': 241, 'Neamt': 234,
    'Oradea': 380, 'Pitesti': 100, 'RimnicuVilcea': 193, 'Sibiu': 253,
    'Timisoara': 329, 'Urziceni': 80, 'Vaslui': 199, 'Zerind': 374,
}
# Catatan: h_sld(Bucharest) = 0, karena goal selalu memiliki estimasi 0.
# Heuristik ini admissible karena jarak garis lurus TIDAK PERNAH melebihi
# jarak tempuh sesungguhnya di jalan (yang berkelok-kelok).
```

Percobaan 2 — Greedy Best-First Search

Kode Program 6 — Greedy Best-First Search

```
def greedy_best_first(graph, start, goal, h):
    frontier = [(h[start], start)] # diurutkan HANYA oleh h(n), evaluation
                                   # function f(n) = h(n)

    parent = {}
    reached = {start}
    expanded = 0
    while frontier:
        _, node = heapq.heappop(frontier)
        expanded += 1
        if node == goal:
            return reconstruct(parent, goal), expanded
        for neighbor in graph.get(node, {}):
            if neighbor not in reached:
                reached.add(neighbor)
                parent[neighbor] = node
                heapq.heappush(frontier, (h[neighbor], neighbor))
    return None, expanded

path, expanded = greedy_best_first(graph, 'Arad', 'Bucharest', h_sld)
print("Lintasan Greedy :", " -> ".join(path))
print("Node dieksplorasi:", expanded)
```

Output yang Diharapkan:

```
Lintasan Greedy : Arad -> Sibiu -> Fagaras -> Bucharest
Node dieksplorasi: 4
```

Greedy Best-First Search hanya mengeksplorasi 4 node — jauh lebih sedikit dibanding UCS (13 node) — namun lintasan yang ditemukan (Arad-Sibiu-Fagaras-Bucharest) BUKAN lintasan optimal; sesuai Percobaan 4 Minggu 2, lintasan termurah berbiaya 418 melalui Rimnicu Vilcea dan Pitesti.

Percobaan 3 — A* Search

Kode Program 7 — A* Search

```
def a_star(graph, start, goal, h):
    frontier = [(h[start], 0, start)] # f(n) = g(n) + h(n)
    parent = {}
    best_cost = {start: 0}
    expanded = 0
    while frontier:
        f, g, node = heapq.heappop(frontier)
        if g > best_cost.get(node, float('inf')):
            continue
        expanded += 1
        if node == goal:
            return reconstruct(parent, goal), g, expanded
        for neighbor, step_cost in graph.get(node, {}).items():
            new_g = g + step_cost
            if new_g < best_cost.get(neighbor, float('inf')):
                best_cost[neighbor] = new_g
                parent[neighbor] = node
                heapq.heappush(frontier, (new_g + h[neighbor], new_g, neighbor))
    return None, None, expanded

path, cost, expanded = a_star(graph, 'Arad', 'Bucharest', h_sld)
```

```
print("Lintasan A* :", " -> ".join(path))
print("Total biaya :", cost)
print("Node dieksplorasi:", expanded)
```

Output yang Diharapkan:

```
Lintasan A* : Arad -> Sibiu -> RimnicuVilcea -> Pitesti -> Bucharest
Total biaya : 418
Node dieksplorasi: 6
```

A* menemukan lintasan optimal yang sama persis dengan UCS (biaya 418), tetapi hanya mengeksplorasi 6 node dibandingkan 13 node pada UCS — inilah bukti nyata bahwa heuristik yang baik membuat A* jauh lebih efisien tanpa mengorbankan optimalitas.

B.5 Tabel Perbandingan Lima Algoritma

Lengkapi tabel berikut menggunakan hasil dari Minggu 2 (BFS, DFS, UCS) dan Minggu 3 (Greedy, A*) untuk kasus Arad → Bucharest.

Algoritma	Lintasan	Total Biaya	Node Dieksplorasi	Optimal?
BFS				
DFS				
UCS				
Greedy Best-First				
A*				

B.6 Eksperimen Mandiri

1. Uji Greedy Best-First Search dan A* untuk pasangan kota lain, misalnya Iasi → Fagaras dan Neamt → Pitesti. Catat hasilnya pada tabel serupa B.5.
2. Buat heuristik baru yang sengaja TIDAK admissible, misalnya $h_{\text{tidak_admissible}} = \{\text{kota: nilai} \times 2 \text{ for kota, nilai in } h_{\text{sld.items()}}\}$, lalu jalankan A* dengan heuristik tersebut untuk kasus Arad → Bucharest. Apakah lintasan yang ditemukan masih optimal (biaya 418)? Jelaskan mengapa demikian.
3. Implementasikan Weighted A* Search dengan evaluation function $f(n) = g(n) + W \times h(n)$ untuk $W = 1.5$ dan $W = 2$. Bandingkan jumlah node yang dieksplorasi dan total biaya solusi dengan A* biasa ($W = 1$) pada kasus Arad → Bucharest.
4. (Tantangan — Studi Kasus 8-Puzzle) Implementasikan A* Search untuk masalah 8-puzzle menggunakan heuristik Manhattan Distance (total jarak horizontal + vertikal setiap ubin dari posisi saat ini ke posisi tujuannya). Representasikan state sebagai tuple 9 angka, dengan angka 0 mewakili petak kosong.

B.7 Pertanyaan Analisis

1. Mengapa lintasan yang ditemukan Greedy Best-First Search pada Percobaan 2 tidak optimal, padahal jumlah node yang dieksplorasi lebih sedikit dibandingkan A*? Kaitkan jawaban Anda dengan evaluation function masing-masing algoritma.

2. Bandingkan jumlah node yang dieksplorasi oleh UCS (Minggu 2) dan A* (Minggu 3) untuk mencapai solusi optimal yang sama (biaya 418). Apa yang menyebabkan perbedaan efisiensi tersebut?

3. Jelaskan hubungan antara admissibility heuristik dengan jaminan cost-optimal pada A* Search, kaitkan jawaban Anda dengan hasil Eksperimen Mandiri nomor 2.

4. Pada aplikasi navigasi nyata seperti Google Maps atau Waze, mengapa algoritma sejenis A* lebih disukai dibandingkan uniform-cost search murni?
